

Normal flow, flexbox and grid, and how to choose between a narrow reading column and a multi-column information layout.

```
<!-- viewport -->  
<meta content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no" />  
  
-->  
<link href="/favicon.ico" type="image/x-icon" />  
<link href="/favicon.ico" type="image/x-icon" />  
  
-->  
<link rel="stylesheet" href="css/materialize.min.css" media="screen, projection" />  
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/font-awesome/4.6.0/css/font-awesome.min.css" />  
<link rel="stylesheet" href="/css/animate.css" />  
<link rel="stylesheet" href="css/theme.css" />  
  
-->  
<div class="banner">  
  <div class="nav">  
    <div class="nav-wrapper">  
      <a href="#" class="brand-logo hide-on-med-and-up"><span style="font-weight: bold"></span></a>  
    </div>  
  </div>  
</div>  
  
-->  
<div class="container">  
  <div class="row">  
    <div class="col m8 m8 18 hide-on-small-only">starter</div>  
    <div class="col m4 m4 18 hide-on-small-only">network and workspace tool for runners, it's advanced network mapping tool</div>  
  </div>  
</div>
```

Technical outcomes

By the end of this lecture you can build, debug and choose between the layout systems CSS provides.

Predict

Trace how normal flow lays out block and inline boxes before any layout property is applied.

Build

Write working flexbox and grid layouts from real CSS properties, not memorised snippets.

Choose

Justify flexbox vs grid, and a single reading column vs a multi-column layout, for a given piece of content.

Before any layout property runs

Every element is already laid out by normal flow — layout CSS changes that default, it does not create it.

Normal flow places block-level boxes (p, div, section, h1–h6) one below another, each taking the full available width. It places inline boxes (span, a, strong, img by default) left-to-right within a line, wrapping to a new line only when the current one is full.

Block boxes

display: block. Stack vertically. Width defaults to 100% of the containing block; height defaults to the height of their content.

Inline boxes

display: inline. Flow horizontally inside a line box. Cannot take an explicit width or height; top/bottom margin is ignored.

Test it: block vs inline, not by eye

A testable check you can run in any browser, not a visual guess.

```
<style>
  span.demo { background: gold; width: 300px; }
  div.demo  { background: teal;  width: 300px; }
</style>
<span class="demo">A span ignores width</span>
<div class="demo">A div fills the 300px width</div>
```

Testable claim: set width: 300px on both. The div's rendered box measures 300px in DevTools' computed-style panel. The span's rendered box still measures only as wide as its text — width on an inline box is not applied.

display changes the box type, not the element

Any element can be told to behave like a different box type.

Value	Effect
inline-block	Flows inline with siblings but accepts width, height and vertical margin
flex	Turns this element into a flex container; its direct children become flex items
grid	Turns this element into a grid container; its direct children become grid items
none	Removes the box entirely — it takes no space and is not in the accessibility tree

Flexbox: one axis at a time

flex-direction sets the main axis; everything else in flexbox is described relative to it.

```
.nav {  
  display: flex;  
  flex-direction: row; /* main axis runs  
left→right */  
}  
.stack {  
  display: flex;  
  flex-direction: column; /* main axis runs  
top→bottom */  
}
```

Main axis

The direction flex-direction points. justify-content aligns items along it.

Cross axis

Perpendicular to the main axis. align-items aligns items along it.

Building a real navigation bar

A working, testable flex layout — not a description of one.

```
.site-nav {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 1.5rem;  
  padding: 0.75rem 1.5rem;  
}  
.site-nav__links { display: flex; gap: 1rem; }
```

align-items: center

Vertically centres the logo and links even if one is taller than another.

justify-content: space-between

Pushes the logo to the start and the link group to the end, with space distributed between them.

gap replaced margin hacks

gap adds space only between flex/grid items — never at the outer edge.

Before gap existed, spacing children required margin-right on every item except the last (:not(:last-child)) or negative-margin tricks on the container. gap: 1rem on a flex or grid container does the same job with one line and no selector gymnastics.

```
.site-nav__links > * + * { margin-left: 1rem; } /* old way */  
.site-nav__links { display: flex; gap: 1rem; } /* current way */
```

Test it: inspect either rule in DevTools — both produce the same 1rem visual gap, but only gap survives a change in flex-direction unmodified.

Flex-grow, flex-shrink, flex-basis

The shorthand flex: A B C controls how items share leftover or missing space.

Property	What it controls
flex-basis	Starting size before growing/shrinking is applied (like width, but axis-aware)
flex-grow	Share of extra space this item takes when the container has room to spare
flex-shrink	Share of the deficit this item gives up when the container is too small

```
.card    { flex: 1 1 240px; } /* grow, shrink, from 240px */
.sidebar { flex: 0 0 260px; } /* fixed 260px, never grows/shrinks */
```

flex-wrap and a responsive card row

Without flex-wrap, flex items are compressed, never pushed to a new line.

```
.card-row {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}  
.card-row > * { flex: 1 1 220px; }
```

Test it: shrink the browser window in DevTools' device toolbar. Cards keep a minimum ~220px width and drop to fewer per row, then wrap to a new row entirely below about 660px — with no media query written.

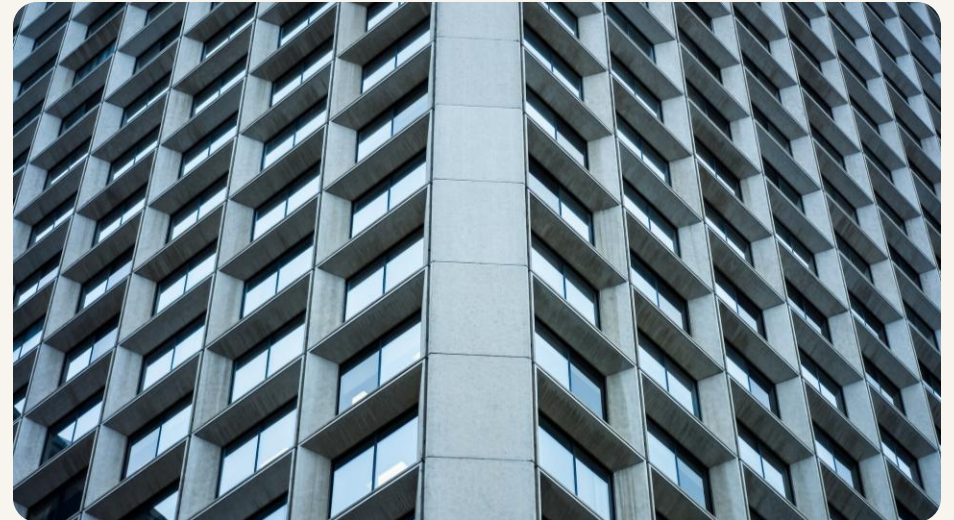


Grid: two axes, defined together

`grid-template-columns` and `grid-template-rows` lay out both dimensions in one declaration.

```
.gallery {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 200px 200px;  
  gap: 1rem;  
}
```

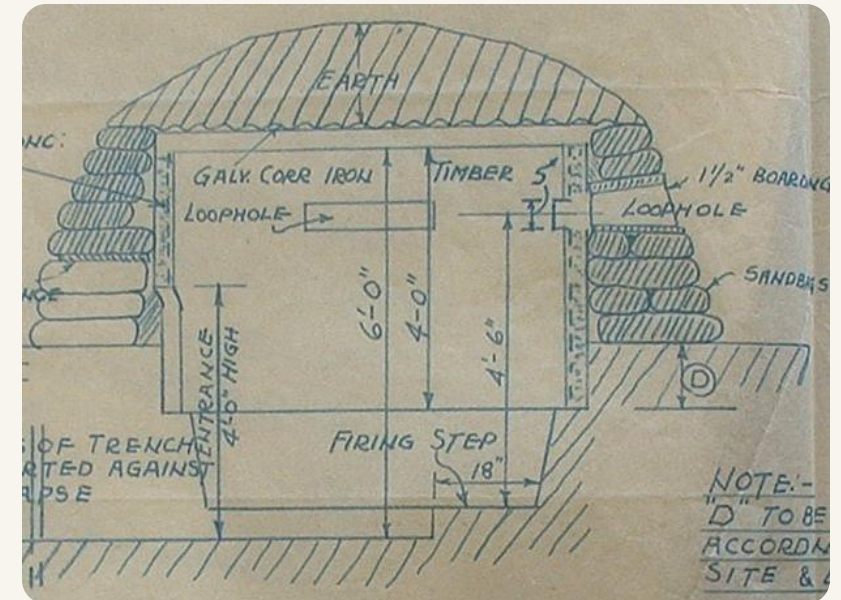
The `fr` unit divides remaining space into fractions after fixed-size tracks are subtracted — three equal `1fr` columns share whatever width the container has.



Naming areas with grid-template-areas

A page layout described visually, in the stylesheet itself.

```
.page {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-areas:
    "sidebar header"
    "sidebar main";
}
.sidebar { grid-area: sidebar; }
.header { grid-area: header; }
.main { grid-area: main; }
```



Auto-fit, auto-fill and minmax: a self-sizing card grid

One line of CSS that adds and removes columns as the viewport changes — no breakpoints.

```
.gallery {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit, minmax(220px, 1fr));  
  gap: 1rem;  
}
```

minmax(220px, 1fr) tells the browser: never shrink a track below 220px, but let it grow to fill remaining space. **auto-fit** collapses empty tracks so the last row of cards stretches instead of leaving a gap.



auto-fit vs auto-fill — the difference that trips people up

Both compute the same track count. They differ only when the grid has fewer items than tracks.



auto-fit



auto-fill



Test it: build a grid with 2 items and repeat(auto-fit/auto-fill, minmax(200px, 1fr)) inside a 900px container. auto-fit gives 2 wide items; auto-fill leaves visible empty tracks beside them.

Flexbox or grid — a decision rule, not a preference

The choice follows from the content's dimensionality, not from habit.

Choose flexbox when

You are arranging items along one dimension and want their size to respond to their content — a nav bar, a button group, a tag list.

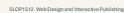
Choose grid when

You are arranging items along two dimensions at once, or need precise alignment between rows and columns — a page skeleton, a card gallery, a form.

Rule of thumb: if you find yourself nesting flex containers to fake rows and columns together, that is the signal to switch to grid.

Line length, not font size, is the main lever for reading comfort in long-form text.

```
.article p {
  max-width: 60ch;
  line-height: 1.6;
}
```

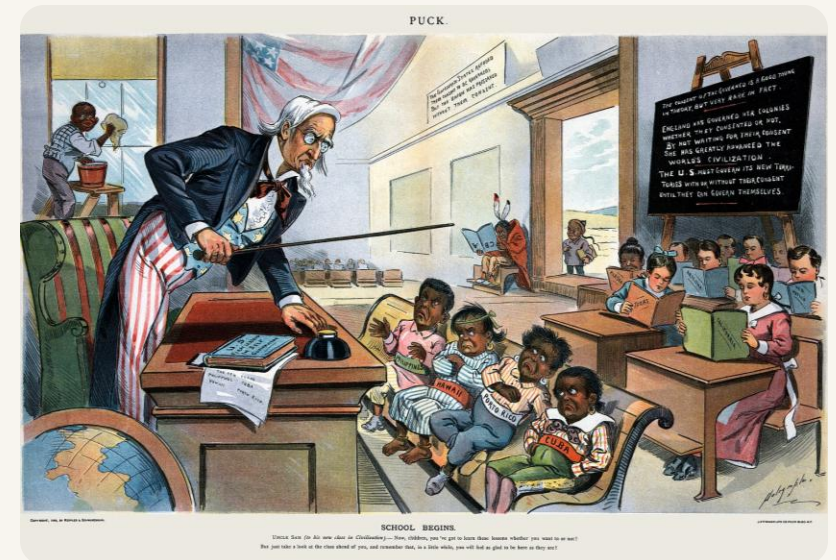


Multi-column layout: column-count and column-width

CSS can lay one flow of text into newspaper-style columns without touching the HTML.

```
.brief {  
  column-width: 22ch;  
  column-gap: 2rem;  
  column-rule: 1px solid #d7d0c4;  
}
```

column-width sets an ideal column size; the browser fits as many columns as fit, then shares leftover width between them — the same responsive logic as auto-fit grid tracks, applied to text flow.



Narrow column vs multi-column: the actual comparison

webModules[3] asks you to compare these directly — here is the testable difference.

Dimension	Single reading column	Multi-column info layout
Reading pattern	One continuous vertical scroll	Zig-zag: down one column, jump to the top of the next
Best for	Long-form narrative prose meant to be read start to finish	Reference or scannable content: glossaries, short news items, indexes
Failure mode if misused	Wastes width on a wide screen, feels empty	A reader loses their place jumping between columns of long paragraphs

Container queries: responding to the container, not the viewport

@container lets a component adapt to the space it actually has, wherever it is placed.

```
.card-wrapper { container-type: inline-size; }  
  
@container (min-width: 380px) {  
  .card { display: grid; grid-template-columns: 120px 1fr; }  
}
```

A media query asks “how wide is the browser window?”. A container query asks “how wide is my parent right now?” — the same .card component can sit in a 300px sidebar or a 900px main column and lay itself out correctly in each.

Layout bug 1 — collapsing margins

Adjacent vertical margins between block siblings do not add; the larger one wins.

```
<div style="margin-bottom: 20px">First</div>
<div style="margin-top: 30px">Second</div>
<!-- gap between them is 30px, not 50px -->
```

The fix

Give the parent display: flex, display: grid, or a non-zero border/padding — any of these opts the parent out of margin collapsing entirely.

Layout bug 2 — the flex item that will not shrink

Flex items default to `min-width: auto`, which can override `flex-shrink` entirely.

```
.row { display: flex; }  
.row > pre { flex: 1; } /* long unbroken text still overflows */  
.row > pre { flex: 1; min-width: 0; } /* fix: now it wraps/scrolls inside */
```

A flex or grid item's default min-width (and min-height, on the cross axis) is auto, meaning “never smaller than my content's intrinsic size.” A long unbroken string like a URL then forces the item wider than flex-shrink allows, and it overflows the row.

Test it: put `min-width: 0` back on the item and the same content wraps inside its column instead of pushing the layout wider.

Layout bug 3 — grid blowout from implicit tracks

Content that does not fit an explicit grid creates new tracks the designer never sized.

```
.gallery { display: grid; grid-template-columns: repeat(3, 1fr); }  
.gallery > .wide { grid-column: span 5; } /* only 3 columns exist! */
```

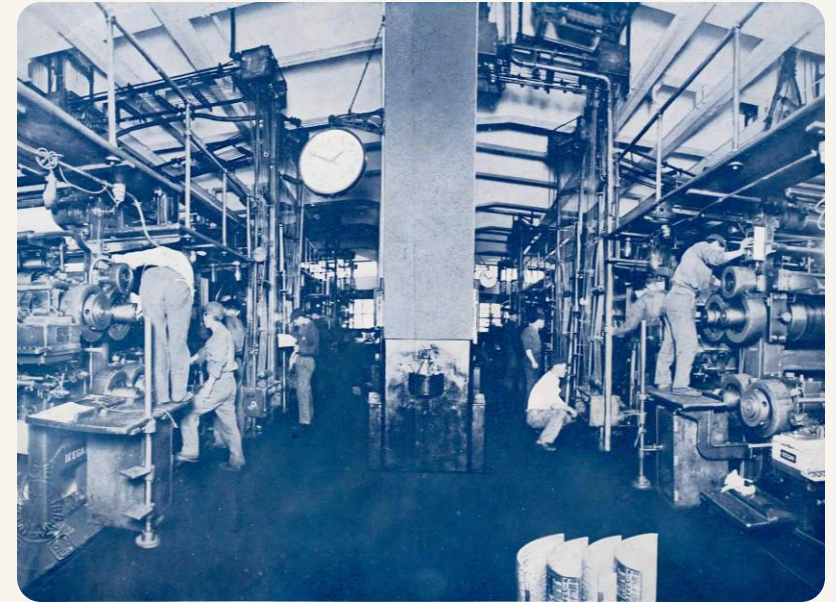
The fix

Constrain implicit tracks explicitly with `grid-auto-columns`, or clamp spans with `min(5, 3)` logic in code before the value reaches CSS, so a span can never exceed the defined column count.

Before / after: refactoring a float-based layout to grid

The historical technique this replaces, and why the replacement is more robust.

```
/* before: float + clearfix */  
.sidebar { float: left; width: 220px; }  
.main    { margin-left: 240px; }  
.row::after { content: ""; display: block; clear: both; }
```



The grid version — same result, no clearfix

grid-template-columns replaces float, width and the clearfix hack in one declaration.

```
/* after: grid */
.row {
  display: grid;
  grid-template-columns: 220px 1fr;
  gap: 1.5rem;
}
.sidebar { /* no float, no width hack needed */ }
```

What improved

The main column's height is no longer independent of the sidebar's; both sit in one grid row and stay aligned even if either one's content changes length.

Debugging layout in DevTools

Both flex and grid have dedicated, official inspector overlays — use them before guessing.

- 1 Select the container element, then click the flex or grid badge next to its display value.
- 2 Toggle the overlay to see track/line numbers and item boundaries drawn directly on the page.
- 3 Open the Layout or Flex panel to read each item's resolved size, not just its declared CSS.

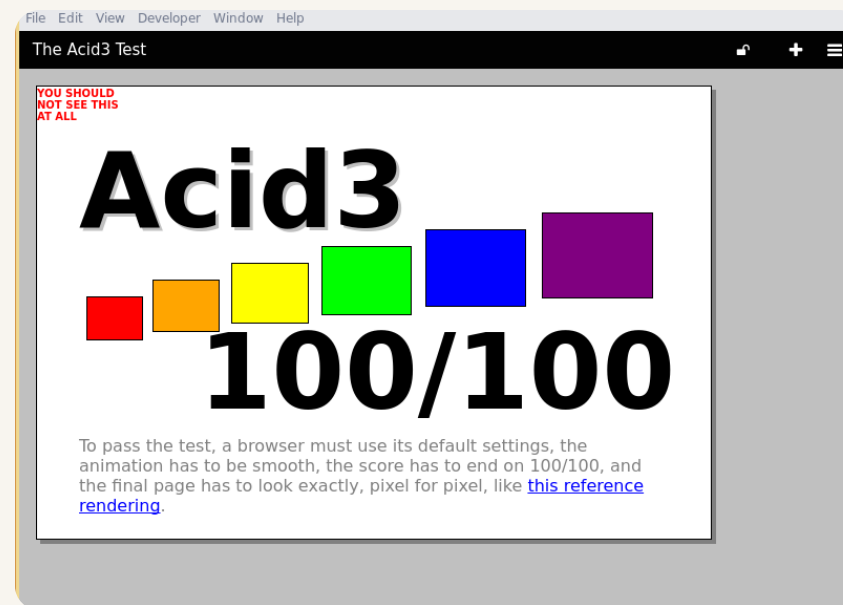


Image sources and licenses

Every photograph in this deck is CC0, public domain, or otherwise unrestricted, sourced from Wikimedia Commons.

File:Key West Citizen newspaper front page 12 June 1908.jpg — Public domain, Florida Keys--Public Libraries

File:School Begins (Puck Magazine 1-25-1899).jpg — Public domain, Louis Dalrymple

File:Norcon blueprint section.jpg — Public domain, HM Government

File:Canadian law books on shelf (31089552820).jpg — CC0, David Whelan

File:Rotary printing press machine, Asahi-Shinbun, 1932.jpg — Public domain, Unknown authorUnknown author

File:Min (web browser) 1.5.1 tests Acid3 screenshot.png — Public domain, Jayvee Enaguas (HarvettFox96)

File:Movie theater seats (Unsplash).jpg — CC0, Elijah Flores baesik

File:Reflective skyscraper windows (Unsplash).jpg — CC0, Redd Angelo reddangelo16

File:CSS code on a screen (Unsplash).jpg — CC0, Sai Kiran Anagani _imkiran

File:Aerial view of the Seattle Center, Seattle, Washington (4860575885).jpg — No restrictions, UW Digital Collections

File:Man with smartphone and laptop (Unsplash).jpg — CC0, Alejandro Escamilla alejandroescamilla



Full description-page URLs are recorded in each slide's speaker notes. All images accessed 13 September 2026.

Studio task

Build both layout systems in the same page and justify each choice in writing.

Brief	Build a page with a flexbox navigation bar (logo left, links right, wraps below 600px) and a CSS Grid card gallery below it (auto-fit, minmax(220px, 1fr)).
Constraints	No fixed pixel widths on the cards. No media queries in the gallery. The nav must not overflow at 320px wide.
Evidence to submit	A screenshot at 375px and 1440px, your CSS, and 100 words justifying flexbox for the nav and grid for the gallery using this lecture's decision rule.

Review checklist

How your studio submission is assessed.

Correctness

Layout matches the brief at both 375px and 1440px without horizontal overflow.

System choice

Flexbox used for the one-dimensional nav; grid used for the two-dimensional gallery.

Robustness

No fixed pixel widths on cards; the gallery re-flows correctly as items are added or removed.

Justification

Written explanation correctly applies the flexbox-vs-grid decision rule to this specific layout.

Further practice and preparation

Before next session.

Flexbox reference

MDN, Basic concepts of flexbox: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox

Grid reference

MDN, Basic concepts of grid layout: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Basic_concepts_of_grid_layout

Container queries

MDN, CSS container queries: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_containment/Container_queries

Bring to Week 5

One page of your studio build printed or screenshotted, ready to critique in session.

Next week

From layout to typography and colour — making the page you can now build easy to read.

Week 4 gave you the structural tools: normal flow as the default, flexbox for one dimension, grid for two, and the reading-column vs multi-column decision for long-form and reference content.

Week 5 preview

Typography, colour and visual hierarchy — how type scale, contrast and whitespace make a correctly laid-out page comfortable to actually read.